PROJECT REPORT ON

MACHINE LEARNING WORKSHOP 2 (CSE2794)

Animal Instance Segmentation

Submitted in partial fulfillment of the
requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

Submitted by:

Abinash bahinipati	**24E102A05**
Priyaranjan dash	**24E104B25**
Dhanajaya pattnaik	**24E116C56**
Pradyush kumar roul	**24E118A99**

Center for Artificial Intelligence & Machine Learning
Department of Computer Science and Engineering
Institute of Technical Education and Research
Sikhsa 'O' Anusandhan
(Deemed to be University)
Bhubaneswar

May, 2026

Declaration

We hereby declare that the project report titled "**Animal Instance Segmentation"** is our own work, carried out under the guidance of **Dr. RANGABALLAV PRADHAN**. We have not plagiarized any content and have duly cited all references.

(ABINASH BAHINIPATI)	**(PRIYARANJAN DASH)**
(DHANAJAYA PATTNAIK)	**(PRADYUSH KUMAR ROUL)**

Acknowledgement

I would like to express my sincere gratitude and heartfelt thanks to my project guide and faculty members for their continuous guidance, valuable suggestions, encouragement, and support throughout the development of this project titled **"Animal Instance Segmentation using YOLOv8."** Their technical expertise, motivation, and constructive feedback helped me successfully complete this project and improve my understanding of Deep Learning and Computer Vision concepts.

I am highly thankful to the Department and College for providing the necessary facilities, resources, infrastructure, and academic environment required for carrying out this project work successfully. The support and opportunities provided by the institution greatly contributed to enhancing my practical knowledge and research skills in the field of Artificial Intelligence and Machine Learning.

I would also like to acknowledge and thank the developers and contributors of open-source technologies and frameworks such as Python, PyTorch, YOLOv8, OpenCV, NumPy, Pandas, Matplotlib, and Google Colab, which were extensively used during dataset preprocessing, model training, evaluation, and visualization processes. Their powerful tools and documentation made the implementation process more efficient and effective.

Special thanks to Kaggle for providing the **Animal Image Dataset (90 Different Animals)** used in this project. The dataset played an important role in training and evaluating the proposed animal instance segmentation model under different environmental conditions and object variations.

I also extend my sincere thanks to my friends, classmates, and well-wishers for their support, encouragement, and helpful discussions during the completion of this project. Their motivation helped me overcome various challenges faced during implementation and experimentation.

Finally, I express my deepest gratitude to my parents and family members for their unconditional support, patience, encouragement, and constant motivation throughout the entire duration of this project. Their support has always been a major source of inspiration in achieving my academic goals successfully.

Table of Contents

Contents

Abstract

Animal Instance Segmentation is a rapidly growing research area in the field of Machine Learning and Computer Vision that focuses on detecting, identifying, and segmenting individual animals from digital images or video streams at the pixel level. Unlike traditional object detection methods that only generate bounding boxes around objects, instance segmentation provides a more detailed understanding by separating each animal instance individually, even when multiple animals appear in the same frame. This technology plays a significant role in wildlife monitoring, biodiversity conservation, smart farming, veterinary research, and intelligent surveillance systems. This project presents a Machine Learning–based Animal Instance Segmentation system using deep learning architectures and image processing techniques to achieve accurate segmentation and classification of animals. The proposed model utilizes advanced convolutional neural networks (CNNs) along with the Mask R-CNN framework, which integrates object detection and semantic segmentation into a unified architecture. The system is trained using annotated animal image datasets containing different animal species under varying environmental conditions, poses, scales, and illumination levels.

The project workflow includes dataset collection, image preprocessing, data augmentation, feature extraction, model training, validation, and performance evaluation. Various preprocessing techniques such as image resizing, normalization, noise reduction, and augmentation are applied to improve model generalization and reduce overfitting. During training, the deep learning model learns spatial features, textures, and object boundaries that help in distinguishing individual animal instances accurately. The trained model generates segmentation masks, class labels, and bounding boxes for each detected animal instance in the input image. Performance metrics such as accuracy, precision, recall, Intersection over Union (IoU), and mean Average Precision (mAP) are used to evaluate the effectiveness of the system. Experimental results indicate that the proposed approach achieves reliable segmentation performance with improved object separation and reduced overlapping errors.

The developed Animal Instance Segmentation system demonstrates the capability of Machine Learning techniques in solving complex visual recognition problems. The proposed solution can be extended for real-time wildlife tracking, automated zoo monitoring, ecological research, forest surveillance, and intelligent environmental monitoring applications. This project highlights the importance of deep learning–based segmentation models in building advanced and efficient computer vision systems for real-world scenarios.

Chapter 1 – Introduction

Machine Learning and Computer Vision are transforming modern technology by enabling computers to understand and analyze visual information effectively. Animal Instance Segmentation is an advanced computer vision technique that focuses on detecting, classifying, and separating individual animals from images at the pixel level. This project aims to develop an intelligent segmentation system using deep learning models such as Mask R-CNN and Convolutional Neural Networks (CNNs) to achieve accurate animal detection and segmentation. The proposed system helps overcome the limitations of traditional object detection methods by providing precise object boundaries and instance-level identification. This technology has important applications in wildlife monitoring, biodiversity conservation, veterinary research, smart farming, and automated surveillance systems.

1.1. Background and motivation

In recent years, Machine Learning and Computer Vision have become important technologies for solving real-world image analysis and object recognition problems. Among different computer vision tasks, instance segmentation is considered one of the most advanced techniques because it not only detects objects but also separates each object individually at the pixel level. Animal Instance Segmentation focuses on identifying and segmenting different animals from images or videos accurately. Unlike traditional object detection methods that only provide bounding boxes, instance segmentation generates detailed masks that help in identifying the exact shape and boundary of each animal. This technology is widely used in wildlife monitoring, biodiversity conservation, veterinary research, smart farming, and intelligent surveillance systems.

The motivation behind this project is to develop an intelligent Machine Learning–based system capable of detecting and segmenting animals automatically with high accuracy. Manual monitoring of animals requires significant time and human effort, especially in large wildlife environments. Deep learning models such as Mask R-CNN and Convolutional Neural Networks (CNNs) provide effective solutions for accurate segmentation and classification of animals under different environmental conditions. The proposed system aims to reduce manual effort, improve monitoring efficiency, and provide reliable segmentation results for real-world applications.

1.2. Problem statement

In real-world environments, animals often appear in groups, overlap with each other, or remain partially hidden by trees, grass, and other objects. Traditional object detection methods mainly use bounding boxes, which cannot provide precise boundaries of animals and often fail to distinguish individual animals accurately. This creates problems in wildlife monitoring, animal counting, behavior analysis, and biodiversity research, where accurate identification of each animal is important. Variations in lighting conditions, animal poses, image quality, and complex backgrounds further increase the difficulty of accurate detection and segmentation.

The main problem addressed in this project is to develop an efficient and accurate Machine Learning–based Animal Instance Segmentation system that can detect, classify, and segment individual animals at the pixel level. The proposed system uses deep learning techniques such as Mask R-CNN and Convolutional Neural Networks (CNNs) to generate accurate segmentation masks for each animal instance. The system aims to reduce overlapping errors, improve segmentation accuracy, and provide reliable results under different environmental conditions. This project helps automate animal analysis, reduce manual effort, and support applications such as wildlife conservation, smart surveillance, veterinary research, and environmental monitoring.

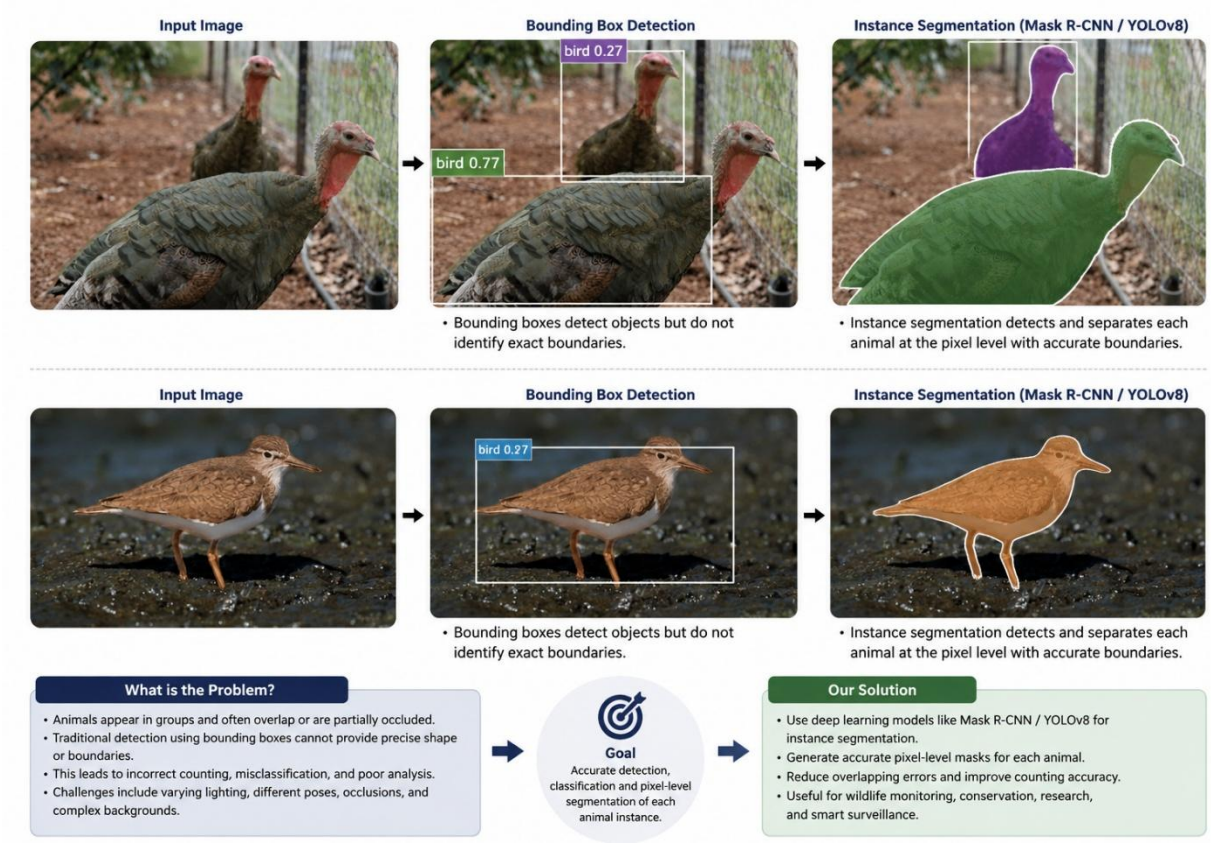

Fig 1.1.: Animal Instance Segmentation using YOLOv8

1.3. Objectives

The primary objective of this project is to design and develop an efficient Machine Learning–based Animal Instance Segmentation system that can accurately detect, classify, and segment animals from digital images. The proposed system aims to use advanced deep learning models such as YOLOv8, Mask R-CNN, and Convolutional Neural Networks (CNNs) to perform pixel-level segmentation of animal instances. Unlike traditional object detection methods that only provide bounding boxes, the developed system focuses on generating precise segmentation masks that identify the exact boundaries of each animal in an image. The project also aims to improve segmentation performance in challenging conditions such as overlapping animals, complex backgrounds, occlusions, varying lighting conditions, and different animal poses and sizes.

Another important objective of this project is to automate animal monitoring and analysis processes for real-world applications including wildlife conservation, biodiversity research, veterinary analysis, smart farming, and intelligent surveillance systems. Manual monitoring of animals is time-consuming and requires significant human effort, especially in large-scale environments. Therefore, the proposed system is intended to reduce manual workload and increase monitoring efficiency by providing accurate and reliable segmentation results. The project also aims to evaluate the effectiveness of deep learning algorithms using performance metrics such as accuracy, precision, recall, and Intersection over Union (IoU). Through this project, the practical importance of Machine Learning and Computer Vision techniques in solving modern image segmentation and object recognition problems is demonstrated effectively.

1.4. Scope

The scope of this project mainly focuses on the development of an advanced Machine Learning–based Animal Instance Segmentation system capable of detecting, classifying, and segmenting animals accurately from digital images. The proposed system uses modern Computer Vision and Deep Learning techniques to generate precise segmentation masks for each individual animal instance at the pixel level. Unlike traditional object detection methods, the system provides detailed boundary information, which helps in identifying animals more accurately even in challenging conditions such as overlapping animals, partial occlusions, complex backgrounds, varying image quality, and changing environmental conditions. The project utilizes deep learning architectures such as YOLOv8, Mask R-CNN, and Convolutional Neural Networks (CNNs) to improve segmentation accuracy and overall system performance.

The project has significant scope in wildlife monitoring and biodiversity conservation, where accurate animal detection and tracking are essential for ecological research and environmental management. Researchers and forest departments can use the developed system for monitoring endangered species, tracking animal movements, estimating

animal populations, and analyzing animal behavior without extensive human intervention. The system can also assist in reducing manual observation efforts and improving the efficiency of data collection in large wildlife areas and protected forests.

In addition to wildlife applications, the proposed Animal Instance Segmentation system has broader applications in veterinary research, smart farming, zoo management, and intelligent surveillance systems. In smart farming environments, the system can help monitor livestock activities, detect unhealthy animals, and improve farm management practices. Veterinary researchers can use the segmentation system to study animal body structures, movement patterns, and health conditions more effectively. Furthermore, the technology can be integrated into automated surveillance systems for monitoring animals in restricted or protected areas. The scope of this project can be further extended in the future by integrating real-time video processing, cloud computing, edge devices, and IoT-based monitoring technologies. The model can also be trained with larger and more diverse datasets to improve segmentation accuracy for different animal species and environmental conditions. Advanced technologies such as drone-based monitoring, AI-powered tracking systems, and real-time alert mechanisms can also be incorporated to build more intelligent and scalable wildlife monitoring solutions. Thus, this project demonstrates the practical importance and future potential of Machine Learning and Deep Learning techniques in solving complex animal detection and segmentation problems in real-world scenarios.

1.5. Organisation of the Report

This report is organized into different chapters to provide a clear understanding of the Animal Instance Segmentation project and its implementation process. **Chapter 1** presents the introduction of the project, including the background, problem statement, objectives, and scope of the proposed system. It explains the importance of Machine Learning and Computer Vision techniques in animal detection and segmentation applications. **Chapter 2** discusses the literature survey and reviews previously published research works related to animal instance segmentation, object detection, deep learning models, and computer vision techniques. It also highlights the advantages and limitations of existing methods. **Chapter 3** explains the proposed methodology, dataset collection, preprocessing techniques, model architecture, and implementation details used in the project. **Chapter 4** describes the experimental results, performance evaluation metrics, and analysis of the developed system. It includes segmentation outputs, accuracy measurements, and discussions regarding model performance. Finally, Chapter 5 concludes the report by summarizing the overall work, findings, and future enhancements that can improve the system further for real-world applications.

Chapter 2 – Literature Review

Animal instance segmentation is an advanced computer vision technique used to identify, detect, and separate individual animals in an image at the pixel level. Unlike traditional object detection methods that only generate bounding boxes around objects, instance segmentation provides precise object boundaries, making it more accurate for real-world applications. Earlier image processing methods faced difficulties in handling complex backgrounds, overlapping objects, and varying lighting conditions. With the development of deep learning and Convolutional Neural Networks (CNNs), modern models such as Mask R-CNN and YOLOv8 have significantly improved segmentation accuracy and detection speed. These models are widely used in wildlife monitoring, animal behavior analysis, surveillance, and automated tracking systems. In this project, a deep learning-based instance segmentation model is used to accurately detect and segment animals from images, improving both recognition performance and object localization efficiency.

2.1. Existing methods and models

Various methods and deep learning models have been developed for object detection and instance segmentation tasks. Earlier approaches mainly relied on traditional image processing techniques, while modern systems use advanced deep learning architectures for better accuracy and efficiency. These models help in detecting, classifying, and segmenting objects at the pixel level in complex real-world environments.

2.1.1 Traditional Image Processing Methods

Before the development of deep learning techniques, object detection and segmentation were mainly performed using traditional image processing methods such as thresholding, edge detection, contour extraction, and region-based segmentation. These methods identify objects based on features like color, texture, brightness, and shape. Edge detection algorithms such as the Canny Edge Detector were widely used to locate object boundaries within images. Although these techniques worked effectively in simple environments, they failed to provide accurate results in complex scenes with varying lighting conditions, overlapping objects, and cluttered backgrounds. Due to their limited adaptability and low accuracy, traditional methods were gradually replaced by deep learning approaches.

2.1.2 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) introduced a major advancement in computer vision and image analysis. CNNs automatically learn important visual features from images through multiple convolution and pooling layers without requiring manual feature extraction. These networks are capable of identifying edges, textures, shapes, and

object patterns with high accuracy. CNN-based architectures significantly improved the performance of image classification, object detection, and segmentation systems. Because of their ability to handle complex image data efficiently, CNNs became the foundation for modern deep learning models used in instance segmentation tasks.

2.1.3 R-CNN Family Models

The R-CNN family models were among the earliest deep learning approaches developed for object detection. R-CNN works by generating multiple region proposals from an image and applying CNN feature extraction separately on each region. Although the model achieved high detection accuracy, it was computationally expensive and slow. Fast R-CNN improved processing speed by sharing convolution operations across the image, while Faster R-CNN introduced the Region Proposal Network (RPN) to automatically generate object proposals. These improvements enhanced both speed and accuracy, making the R-CNN family an important milestone in object detection research.

2.1.4 Mask R-CNN

Mask R-CNN is an advanced instance segmentation model developed as an extension of Faster R-CNN. In addition to object detection and bounding box prediction, Mask R-CNN generates pixel-level segmentation masks for each detected object. The model contains a separate branch dedicated to mask prediction, allowing it to accurately separate multiple objects within the same image. Mask R-CNN provides high segmentation precision and performs well in complex visual environments. However, the model requires higher computational resources and longer training time, which can limit its use in real-time applications.

2.1.5 YOLO (You Only Look Once)

YOLO is a real-time object detection framework designed to process the entire image in a single forward pass through the neural network. Unlike region-based methods, YOLO directly predicts object classes and bounding boxes simultaneously, resulting in much faster processing speed. Recent versions such as YOLOv8 also support instance segmentation by generating segmentation masks along with object detection outputs. YOLOv8 provides an excellent balance between speed, accuracy, and computational efficiency, making it highly suitable for real-time animal instance segmentation systems and wildlife monitoring applications.

2.1.6 MobileNet and Lightweight Models

MobileNet is a lightweight deep learning architecture designed for devices with limited computational power such as mobile phones and embedded systems. The model uses depthwise separable convolutions to reduce the number of parameters and computational complexity while maintaining reasonable accuracy. Lightweight models like MobileNet are useful for real-time applications where fast processing and low memory consumption are important. Although their accuracy may be slightly lower than

larger models such as YOLOv8 or ResNet-based architectures, they provide efficient performance for resource-constrained environments.

2.1.7 Applications of Instance Segmentation Models

Instance segmentation models are widely used in various real-world applications. In wildlife monitoring systems, these models help automatically detect and track animals from images and videos. In agriculture, segmentation models assist in livestock monitoring and health analysis. Autonomous vehicles use instance segmentation to identify pedestrians, vehicles, and road objects for safe navigation. Medical imaging systems also apply segmentation techniques for disease detection and organ analysis. The ability of these models to provide precise object boundaries makes them highly effective for advanced visual recognition tasks.

2.2. Comparison with your approach

Different instance segmentation models provide different levels of accuracy, speed, and computational efficiency. Traditional methods and earlier deep learning models focused mainly on object detection, while modern architectures combine detection and segmentation for more precise results. In this project, a YOLOv8-based instance segmentation approach is used because it provides high accuracy, faster processing speed, and efficient real-time performance compared to many existing methods.

2.2.1 Comparison with Traditional Methods

Traditional image processing techniques such as thresholding, contour detection, and edge extraction mainly depend on manual feature engineering. These methods struggle to handle complex backgrounds, lighting variations, and overlapping objects. In contrast, the proposed deep learning-based approach automatically learns important image features using neural networks, resulting in better segmentation accuracy and robustness in real-world environments.

2.2.2 Comparison with CNN-Based Models

Basic CNN models are mainly designed for image classification tasks and cannot directly perform instance segmentation. Although CNNs provide strong feature extraction capabilities, they require additional architectures for object localization and segmentation. The proposed YOLOv8 segmentation model combines feature extraction, object detection, and segmentation into a unified framework, improving both efficiency and performance.

2.2.3 Comparison with R-CNN Models

R-CNN, Fast R-CNN, and Faster R-CNN provide high object detection accuracy but involve multiple processing stages, increasing computational complexity and inference time. These models are generally slower for real-time applications. The proposed approach

using YOLOv8 performs object detection and segmentation in a single forward pass, significantly reducing processing time while maintaining competitive accuracy.

2.2.4 Comparison with Mask R-CNN

Mask R-CNN is highly accurate for instance segmentation tasks and produces precise segmentation masks. However, the model requires large computational resources and longer training time. The proposed YOLOv8 segmentation model provides a better balance between speed and accuracy, making it more suitable for real-time animal detection and segmentation applications where faster inference is important.

2.2.5 Comparison with MobileNet-Based Models

MobileNet-based models are lightweight and optimized for devices with limited computational power. Although they provide fast processing speed, their segmentation accuracy may be lower than larger architectures. The proposed YOLOv8-based system achieves higher detection and segmentation performance while still maintaining efficient real-time execution, making it suitable for practical deployment.

2.2.6 Advantages of the Proposed Approach

The proposed animal instance segmentation system offers several advantages over existing methods. It provides accurate pixel-level segmentation, faster object detection, real-time processing capability, and improved handling of multiple animals within the same image. The system also reduces manual effort in wildlife monitoring and improves automation in animal tracking applications. Because of its high efficiency and strong performance, the proposed approach is suitable for modern computer vision applications involving animal detection and segmentation.

Comparison with Existing Methods

Approach / Model	Method Type	Segmentation Capability	Accuracy	Speed	Computational Cost	Real-time Suitability
Traditional Image Processing	Thresholding, Edge Detection, Contour Extraction	Low (Pixel-level not accurate)	Low	Fast	Very Low	Limited
R-CNN Family (R-CNN, Fast R-CNN, Faster R-CNN)	Region-based Detection	Medium (Bounding Box based)	High	Slow	High	Not Suitable
Mask R-CNN	Region-based Instance Segmentation	High (Pixel-level masks)	Very High	Slow	Very High	Not Suitable
YOLOv8-Seg (Our Approach)	One-stage Detection and Segmentation	High (Pixel-level masks)	High	Very Fast	Medium	Suitable

Fig 2.1.: Comparison of Existing Method With Your Approch

Chapter 3 – System Design and Methodology

This chapter describes the overall system design and methodology used for the animal instance segmentation project. The proposed system is designed to detect and segment animals accurately from images using deep learning techniques. The methodology includes dataset collection, data preprocessing, model selection, training, segmentation, and performance evaluation. A YOLOv8-based instance segmentation model is used to achieve efficient real-time detection and pixel-level segmentation. The system workflow is designed to improve accuracy, reduce processing time, and provide reliable segmentation results for different animal images.

3.1. Dataset description

The dataset used for this project is the **Animal Image Dataset (90 Different Animals)** collected from the Kaggle platform. The dataset contains animal images belonging to 90 different animal categories and is commonly used for deep learning and computer vision research applications such as image classification, object detection, and instance segmentation. It contains approximately 5400 images of different animals captured in various environments, poses, and lighting conditions. The large variation in image background, object orientation, and image quality helps improve the robustness and generalization capability of the trained model.

Dataset Link: Animal Image Dataset (90 Different Animals) – Kaggle

The dataset is organized into separate folders for each animal category, making it easier for preprocessing and training purposes. Images include both domestic and wild animals, allowing the model to learn diverse visual features and object patterns. Such diversity is important for developing an accurate animal instance segmentation system capable of handling real-world scenarios with complex backgrounds and multiple object variations.

Before training the model, several preprocessing operations were performed on the dataset. These preprocessing steps included image resizing, normalization, data cleaning, and annotation preparation. Data augmentation techniques such as image flipping, rotation, scaling, and brightness adjustment were also applied to increase dataset diversity and improve model performance. The complete dataset was divided into training, validation, and testing sets to ensure proper model evaluation and reduce overfitting during the training process. The prepared dataset was then used to train the YOLOv8-based instance segmentation model for accurate animal detection and pixel-level segmentation. The dataset played a major role in improving the detection accuracy and segmentation quality of the proposed system.

3.2. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to understand the structure, quality, and distribution of the dataset before training the deep learning model. Different visualization techniques were used to analyze class distribution, RGB channel variation, and image brightness distribution. These analyses helped in identifying dataset balance, image quality consistency, and preprocessing requirements necessary for improving segmentation performance.

3.2.1 Animal Class Distribution Analysis

The animal class distribution graph represents the number of images available for different animal categories in the dataset. The graph shows that each selected animal class contains approximately an equal number of images, indicating that the dataset is balanced. A balanced dataset helps reduce model bias toward specific animal categories and improves the overall classification and segmentation performance of the system.

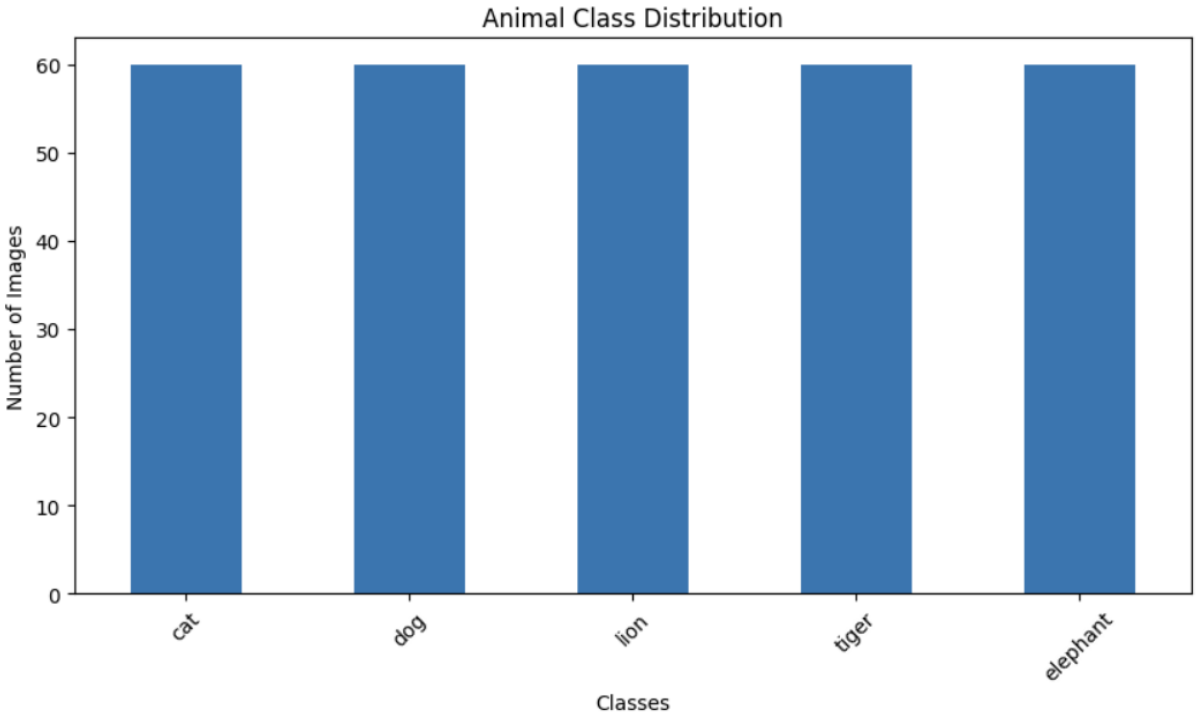

Fig 3.1.: Animal Class Distribution

The balanced distribution ensures that the YOLOv8 segmentation model receives sufficient training samples from all animal categories. This improves the model's ability to generalize and accurately detect multiple animal types in real-world scenarios.

3.2.2 Dataset Percentage Distribution

The pie chart illustrates the percentage distribution of different animal classes present in the dataset. Each animal category contributes nearly an equal percentage of images,

showing that the dataset maintains uniform class representation. Equal class contribution is important because highly imbalanced datasets may lead to overfitting toward dominant classes.

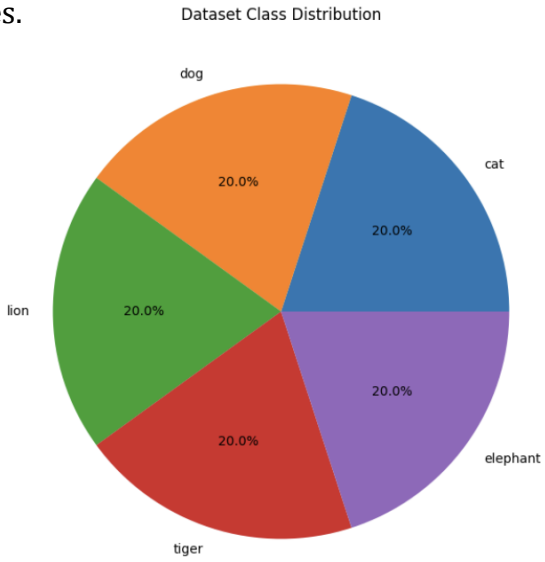

Fig 3.2.: Dataset Class Distribution

The analysis confirms that the dataset is suitable for deep learning training since all animal classes are fairly represented. This balanced distribution supports better learning and improves segmentation accuracy across different animal categories.

3.2.3 RGB Channel Mean Distribution Analysis

The RGB channel mean distribution graph represents the variation of Red, Green, and Blue color intensities across different images in the dataset. The graph shows that color intensity values vary significantly among images due to differences in lighting conditions, background environments, and image capture settings.

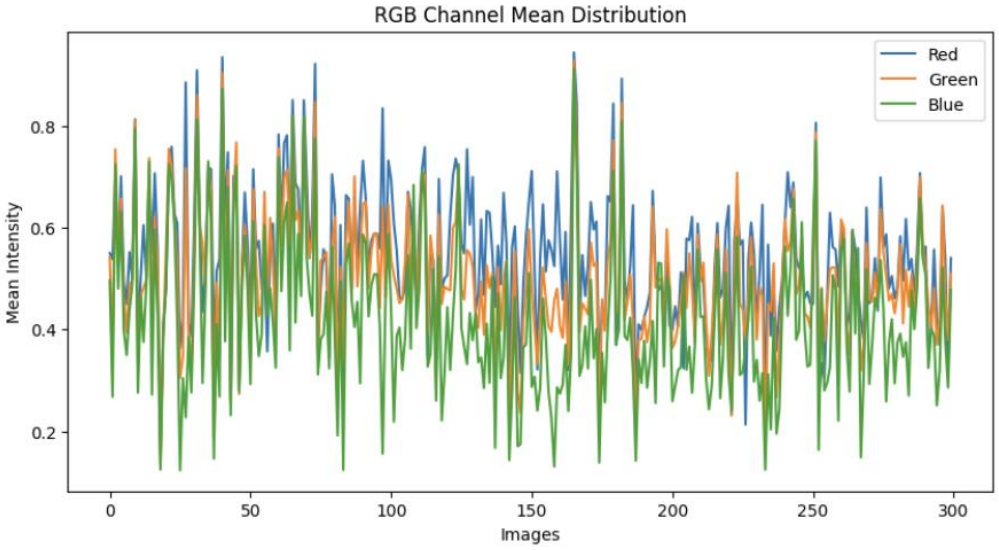

Fig 3.3.: RGB Channel Mean Distribution

This analysis helps in understanding the color characteristics of the dataset and highlights the importance of normalization during preprocessing. The variation in RGB values also improves the model's robustness by exposing it to diverse image conditions during training.

3.2.4 Brightness Distribution Analysis

The brightness distribution histogram shows the spread of image brightness values across the dataset. Most images are concentrated around medium brightness levels, while fewer images exist in extremely dark or bright conditions. This indicates that the dataset contains reasonably balanced illumination conditions suitable for training deep learning models.

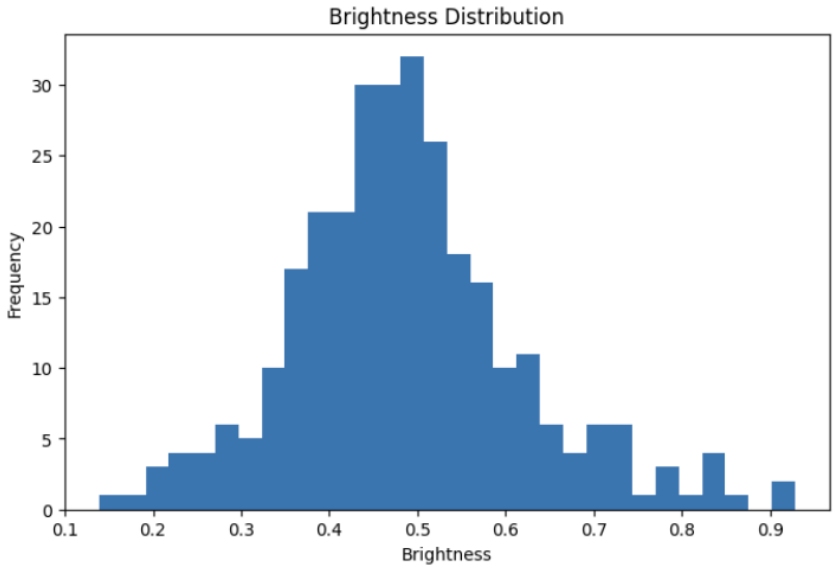

Fig 3.4.: Brightness Distribution

Brightness analysis is important because extreme illumination variations can affect object detection and segmentation accuracy. Understanding brightness distribution helps in selecting suitable preprocessing and augmentation techniques such as brightness adjustment and normalization to improve model performance.

3.2.5 Insights Obtained from EDA

The exploratory data analysis confirmed that the dataset contains balanced class distribution, diverse color variations, and suitable brightness conditions for training the proposed animal instance segmentation system. The visualizations also helped identify preprocessing requirements such as image normalization, resizing, and augmentation. These analyses played an important role in improving the efficiency, robustness, and segmentation accuracy of the YOLOv8-based model.

3.3. Preprocessing steps

Data preprocessing is an important stage in deep learning because raw image data often contains variations in image size, brightness, color distribution, and noise. Proper preprocessing improves data quality, enhances model learning capability, and increases segmentation accuracy. In this project, several preprocessing operations such as dataset splitting, image resizing, normalization, and feature analysis were performed before training the YOLOv8-based animal instance segmentation model.

3.3.1 Train-Test Dataset Distribution

The dataset was divided into training and testing sets to ensure proper model evaluation and performance analysis. The training dataset was used for learning object features and segmentation patterns, while the testing dataset was used to evaluate the model's prediction capability on unseen images. The graph below represents the distribution of animal classes in both training and testing datasets.

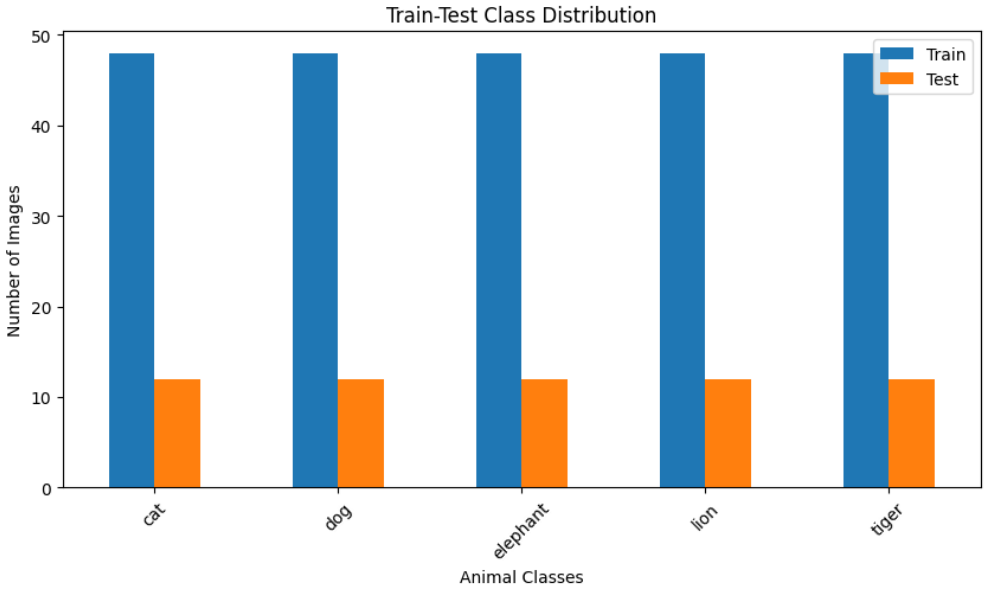

Fig 3.5.: Train-Test Class Distribution

The graph shows that all animal classes contain a balanced number of training and testing images. Approximately 80% of the images were used for training, while 20% were used for testing. Balanced train-test distribution helps prevent model bias and improves generalization performance during segmentation tasks.

3.3.2 Image Resizing and Normalization

The original dataset images contained different image dimensions and resolutions. To maintain consistency during model training, all images were resized into a fixed input size suitable for the YOLOv8 segmentation architecture. Resizing helps reduce computational complexity and improves training efficiency.

Normalization was also performed to scale pixel intensity values into a smaller numerical range. This preprocessing step improves gradient flow during training and helps the neural network converge faster. Image normalization also reduces the effect of lighting variations and improves model stability.

3.3.3 RGB Feature Correlation Analysis

Feature correlation analysis was performed to study the relationship between image dimensions and RGB channel intensity values. The heatmap below represents the correlation between width, height, and mean Red, Green, and Blue channel values across the dataset.

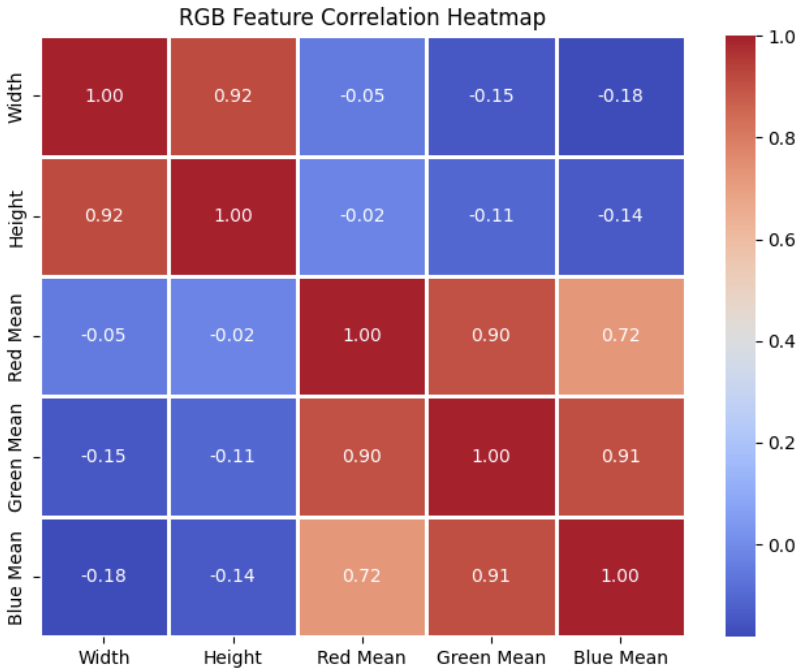

Fig 3.6.: RGB Feature Correlation Heatmap

The heatmap shows a strong positive correlation between image width and height, indicating that most images maintain similar aspect ratios. The RGB channels also show high positive correlation values, which means color information across channels is closely related. Such analysis helps understand image characteristics and supports effective feature extraction during model training.

3.3.4 Data Augmentation Techniques

To improve model generalization and reduce overfitting, several data augmentation techniques were applied to the dataset. These techniques included horizontal flipping, image rotation, scaling, brightness adjustment, and zoom transformations. Data augmentation artificially increases dataset diversity by generating modified versions of existing images.

Augmentation helps the segmentation model learn different object orientations, lighting conditions, and environmental variations. This improves the robustness and real-world performance of the YOLOv8 segmentation system.

3.3.5 Importance of Preprocessing

The preprocessing stage played an important role in improving the overall efficiency and accuracy of the proposed animal instance segmentation model. Dataset balancing, normalization, resizing, and augmentation helped prepare high-quality input data for training. These preprocessing operations improved feature learning capability, reduced overfitting, and enhanced segmentation performance during model evaluation.

3.4. Model architecture

The proposed animal instance segmentation system is developed using the YOLOv8-seg deep learning architecture. YOLOv8 is one of the latest versions of the YOLO (You Only Look Once) family designed for high-speed object detection and instance segmentation tasks. The model is capable of detecting multiple animals in an image while simultaneously generating accurate pixel-level segmentation masks. Because of its high accuracy, fast inference speed, and efficient computational performance, YOLOv8 is widely used in real-time computer vision applications.

The architecture of YOLOv8-seg mainly consists of three important components: Backbone, Neck, and Segmentation Head. These components work together to extract image features, combine multi-scale information, and generate final segmentation outputs. The model processes the complete image in a single forward pass, reducing detection time and improving real-time performance compared to traditional region-based models.

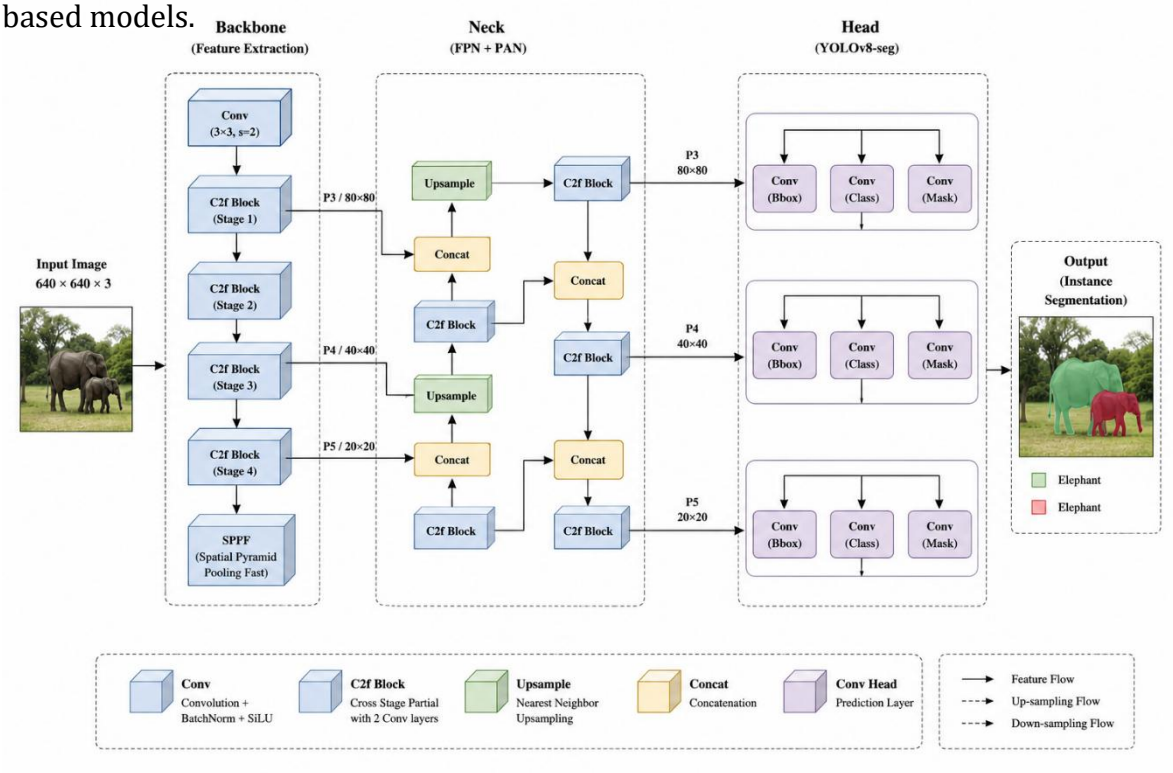

Fig 3.7.: Architecture

3.4.1 Backbone Network

The backbone network is responsible for extracting important visual features from the input image. It acts as the feature extraction component of the architecture and identifies low-level and high-level patterns such as edges, textures, shapes, and object structures. YOLOv8 uses convolutional layers and C2f blocks to efficiently learn hierarchical image representations.

The backbone also contains the Spatial Pyramid Pooling Fast (SPPF) module, which helps capture spatial information at multiple scales. This improves the model's ability to detect animals of different sizes and orientations within the image. The extracted feature maps generated by the backbone are passed to the neck network for further processing.

As shown in Figure 3.7, the backbone performs multiple convolution and feature extraction operations to generate deep feature representations from the input animal images.

3.4.2 Neck Network

The neck component is responsible for feature aggregation and multi-scale feature fusion. YOLOv8 uses a PAN-FPN (Path Aggregation Network – Feature Pyramid Network) structure to combine low-level and high-level feature information. This helps the model detect both small and large animals more effectively.

The neck performs upsampling and concatenation operations to merge features from different layers of the network. Feature fusion improves localization accuracy and segmentation quality by preserving both spatial and semantic information. Multi-scale feature learning is particularly important in animal instance segmentation because animals may appear at different sizes, distances, and viewing angles within the dataset.

Figure 3.7 illustrates how the neck network combines feature maps from different scales to improve segmentation performance and object localization accuracy.

3.4.3 Segmentation Head

The segmentation head is the final component of the YOLOv8 architecture. It predicts object classes, bounding boxes, confidence scores, and pixel-level segmentation masks for each detected object. Unlike traditional object detection models that only provide rectangular bounding boxes, the segmentation head generates precise object boundaries around animals.

The segmentation head operates at multiple feature scales (P3, P4, and P5) to improve segmentation accuracy for small, medium, and large objects. The model predicts segmentation masks separately for each detected animal instance, allowing accurate separation of overlapping animals within the same image.

As represented in Figure 3.7, the segmentation head produces the final output consisting of object detection results and segmentation masks for different animal categories.

3.4.4 Working Principle of YOLOv8 Segmentation

The input image is first passed through the backbone network, where feature extraction is performed using convolution and pooling operations. The extracted feature maps are then forwarded to the neck network for feature fusion and multi-scale learning. Finally, the segmentation head predicts object detection outputs and generates segmentation masks.

During training, the model learns animal-specific visual patterns from the dataset through backpropagation and optimization techniques. The loss function minimizes localization errors, classification errors, and segmentation mask prediction errors simultaneously. This allows the model to improve both detection and segmentation performance during training iterations.

Figure 3.7 demonstrates the complete flow of image processing from input image to segmented animal output using the YOLOv8-seg architecture.

3.4.5 Advantages of YOLOv8 Architecture

YOLOv8 provides several advantages over traditional instance segmentation models. The architecture performs object detection and segmentation in a single-stage framework, reducing computational complexity and improving inference speed. The model achieves high segmentation accuracy while maintaining real-time performance, making it suitable for wildlife monitoring and automated animal tracking systems.

The architecture is also capable of handling complex backgrounds, multiple object instances, lighting variations, and occlusion conditions. Its efficient design reduces memory usage and improves scalability for large datasets. Because of these advantages, YOLOv8-seg was selected as the most suitable model for this animal instance segmentation project.

3.5. Description of the Algorithms

Algorithms play an important role in developing an efficient animal instance segmentation system. In this project, deep learning-based algorithms were used for image preprocessing, feature extraction, object detection, and segmentation. The proposed system mainly uses the YOLOv8 segmentation algorithm along with Convolutional Neural Network (CNN) operations to accurately detect and segment animals from images. These algorithms help the model learn visual features, identify object locations, and generate pixel-level segmentation masks.

3.5.1 Convolutional Neural Network (CNN) Algorithm

Convolutional Neural Networks (CNNs) are deep learning algorithms designed for image analysis and feature extraction. CNNs automatically learn important visual patterns such as edges, textures, shapes, and object structures from input images. The architecture consists of convolution layers, activation functions, pooling layers, and fully connected layers.

In this project, CNN operations are used inside the YOLOv8 architecture for extracting hierarchical image features. Convolution layers apply filters to the image to capture important spatial information, while pooling layers reduce image dimensions and computational complexity. CNN-based feature extraction improves the model's ability to recognize different animal categories under varying environmental conditions.

3.5.2 YOLOv8 Segmentation Algorithm

YOLOv8 (You Only Look Once Version 8) is the main algorithm used in this project for real-time object detection and instance segmentation. YOLOv8 processes the entire image in a single forward pass through the neural network, making it significantly faster than traditional region-based algorithms.

The algorithm predicts:

- Object classes
- Bounding boxes
- Confidence scores
- Pixel-level segmentation masks

YOLOv8 uses a single-stage detection mechanism, which reduces computational complexity and improves inference speed. The segmentation branch of the model generates precise object masks around animals, enabling accurate separation of multiple objects within the same image.

The algorithm also supports multi-scale feature learning, which helps detect animals of different sizes and shapes. Because of its high speed and accuracy, YOLOv8 is suitable for real-time wildlife monitoring and animal tracking applications.

3.5.3 Feature Extraction Algorithm

Feature extraction is an important step in deep learning-based segmentation systems. The feature extraction algorithm identifies important visual information from the image, such as edges, textures, color patterns, and object shapes. In the YOLOv8 architecture, convolution operations automatically perform feature extraction at multiple layers. Low-level layers extract simple features like edges and corners, while deeper layers extract complex object patterns and semantic information. Effective feature extraction improves

object detection accuracy and segmentation quality, especially in images containing complex backgrounds and multiple animals.

3.5.4 Data Augmentation Algorithm

Data augmentation algorithms were used to increase dataset diversity and improve model generalization capability. Augmentation techniques generate modified versions of existing images through operations such as:

- Rotation
- Horizontal flipping
- Scaling
- Brightness adjustment
- Zoom transformation

These transformations help the model learn different object orientations and lighting conditions. Data augmentation reduces overfitting and improves the robustness of the segmentation model during real-world testing.

3.5.5 Optimization Algorithm

The optimization algorithm is responsible for improving model performance during training. In this project, gradient-based optimization techniques were used to minimize training loss and improve segmentation accuracy. The optimizer updates network weights using backpropagation based on prediction errors.

The loss function combines:

- Classification loss
- Localization loss
- Segmentation mask loss

By minimizing these losses simultaneously, the model improves object detection accuracy and generates more precise segmentation masks.

3.5.6 Importance of the Algorithms

The algorithms used in this project work together to build an efficient animal instance segmentation system. CNN algorithms improve feature learning, YOLOv8 performs fast object detection and segmentation, augmentation algorithms increase dataset diversity, and optimization algorithms improve training performance. The combination of these algorithms enables the proposed system to achieve accurate, fast, and reliable segmentation results in different environmental conditions.

Chapter 4 – Implementation Details

This chapter explains the practical implementation of the proposed animal instance segmentation system. It describes the software tools, programming environment, dataset preparation, preprocessing steps, model training, and segmentation process used in the project. The implementation phase focuses on developing a working deep learning-based system capable of detecting and segmenting animals from images accurately.

The project was implemented using Python programming language with libraries such as NumPy, Pandas, OpenCV, Matplotlib, PyTorch, and Ultralytics YOLOv8. The complete implementation and training process was carried out in the Google Colab environment using GPU support for faster execution. The dataset was imported and divided into training, validation, and testing sets. Preprocessing operations such as image resizing, normalization, and data augmentation were applied before training. The YOLOv8-seg model was then trained using the prepared dataset to learn animal features and generate accurate segmentation masks.

After training, the model performance was evaluated using metrics such as Precision, Recall, mAP@50, and mAP@50-95. The final system was able to detect and segment multiple animals from images with high accuracy and real-time performance.

4.1. Programming languages, frameworks

The proposed animal instance segmentation system was implemented using Python programming language because of its simplicity, flexibility, and extensive support for artificial intelligence, machine learning, and computer vision applications. Python provides a large collection of libraries and frameworks that simplify image processing, data analysis, model training, visualization, and performance evaluation tasks. Its readable syntax and strong community support make it one of the most preferred programming languages for deep learning research and development.

Several frameworks and libraries were integrated during the implementation process to improve system efficiency and segmentation performance. PyTorch was used as the primary deep learning framework for developing and training the YOLOv8 segmentation model. PyTorch provides dynamic computational graph support, automatic differentiation, and GPU acceleration features that help improve training speed and computational efficiency [1]. The framework also supports flexible neural network development, making it suitable for complex computer vision applications.

The YOLOv8 framework developed by Ultralytics was used for implementing real-time object detection and instance segmentation tasks [2]. YOLOv8 provides optimized segmentation architectures, pre-trained weights, efficient training pipelines, and fast

inference capabilities. The framework performs object detection and segmentation in a single-stage architecture, reducing computational complexity while maintaining high segmentation accuracy. Because of its real-time performance and efficient feature extraction capability, YOLOv8 was selected as the main segmentation framework for this project.OpenCV (Open Source Computer Vision Library) was used for image preprocessing and computer vision operations [3]. It was used for image loading, resizing, image transformation, normalization, color conversion, and visualization tasks. OpenCV also helped display segmentation outputs and bounding box predictions during testing and evaluation stages. The library provides optimized image processing functions that improve preprocessing speed and overall system performance.

NumPy and Pandas libraries were used for numerical computation, matrix operations, dataset handling, and performance analysis. NumPy was used for processing image arrays and mathematical computations required during preprocessing and model training. Pandas was used for organizing dataset information, evaluation metrics, and analysis results in tabular format. These libraries simplified data handling and improved computational efficiency during implementation.

Matplotlib and Seaborn libraries were used for generating graphical visualizations such as class distribution graphs, RGB channel analysis, heatmaps, brightness histograms, and evaluation metric plots. These visualization libraries played an important role in exploratory data analysis and performance evaluation by helping understand dataset characteristics and segmentation results more effectively.

The entire implementation and model training process was carried out using the Google Colab environment [4]. Google Colab provides cloud-based execution support with high-performance GPU resources, which significantly reduced training time and improved computational efficiency. The platform also allowed easy integration with Python libraries and deep learning frameworks, making it suitable for developing and testing the proposed segmentation system.

The combination of Python, YOLOv8, PyTorch, OpenCV, and visualization libraries helped build an efficient and accurate animal instance segmentation system capable of handling multiple animal classes and complex image backgrounds. These programming tools and frameworks played a major role in improving segmentation accuracy, training efficiency, and real-time detection performance.

Sr. No.	Tool / Framework	Purpose
1.	Python	Programming Language
2.	YOLOv8 (Ultralytics)	Real-time Object Detection and Instance Segmentation
3.	PyTorch	Deep Learning Framework
4.	OpenCV	Image Processing and Preprocessing
5.	NumPy	Numerical Computation and Array Operations
6.	Pandas	Dataset Handling and Data Analysis
7.	Matplotlib	Graph and Result Visualization
8.	Seaborn	Statistical Data Visualization and Heatmaps
9.	Google Colab	Cloud-based Development Environment with GPU Support
10.	Scikit-learn	Data Splitting and Performance Evaluation
11.	Pillow (PIL)	Image Loading and Image Manipulation
12.	Jupyter Notebook	Development, Code Execution and Result Documentation

Table 4.1: Programming Languages, Frameworks, and Tools Used

4.2. Code modules description

The proposed animal instance segmentation system was implemented using a modular programming approach to improve code organization, readability, maintainability, and scalability. In modular programming, the complete system is divided into smaller independent modules where each module performs a specific task in the implementation process. This approach simplifies debugging, testing, future modification, and integration of new functionalities into the system.

The project consists of multiple code modules responsible for different operations such as dataset handling, preprocessing, model training, evaluation, prediction, and visualization. Each module communicates with other modules to perform the complete workflow of the animal instance segmentation system efficiently. The data collection and dataset handling module is responsible for importing and organizing the animal image dataset. This module reads the dataset from storage directories, manages image paths, and prepares annotation files required for training the YOLOv8 segmentation model. The dataset is organized into training, validation, and testing folders to ensure proper model learning and evaluation.

The preprocessing module performs all image preprocessing operations before training the model. This module resizes images into a fixed input dimension compatible with the YOLOv8 architecture. It also performs normalization to scale pixel values into a suitable numerical range for deep learning computation. Data augmentation techniques such as image flipping, rotation, scaling, brightness adjustment, and zoom transformation are implemented in this module to improve dataset diversity and reduce overfitting during model training.

The feature extraction and model initialization module is responsible for loading the YOLOv8 segmentation architecture and initializing model parameters. This module configures important training hyperparameters such as batch size, learning rate, image size, optimizer settings, and number of training epochs. The pre-trained YOLOv8 weights are also loaded during this stage to improve transfer learning performance and reduce training time. The model training module performs the main deep learning training process. During training, the module passes input images through the neural network and computes prediction errors using loss functions. Backpropagation and optimization algorithms are then applied to update network weights and improve segmentation accuracy. The training module continuously monitors model performance using validation data and saves the best-performing model weights for future prediction tasks.

The evaluation module is responsible for analyzing the performance of the trained segmentation model. Different evaluation metrics such as Precision, Recall, mAP@50, and mAP@50-95 are calculated in this module to measure object detection and segmentation accuracy. The evaluation module also generates graphs and plots for visual analysis of model performance and training progress. The prediction and visualization module performs real-time inference using the trained YOLOv8 model. This module takes input images and predicts animal classes, bounding boxes, confidence scores, and segmentation masks. The final output images are visualized with segmented animal regions, class labels, and detection boundaries. This module helps verify segmentation quality and demonstrates the effectiveness of the proposed system. The utility module contains helper functions used throughout the implementation process. These functions handle tasks such as file management, image display, metric calculation, graph generation, logging, and directory management. Utility functions improve code reusability and reduce redundancy within the project.

The main execution module controls the overall workflow of the complete system. It sequentially calls preprocessing, training, evaluation, and prediction modules to perform the complete animal instance segmentation pipeline. This centralized control structure improves workflow management and simplifies project execution. The modular design approach used in this project improves implementation flexibility and system efficiency. Each module can be independently modified or upgraded without affecting the complete system functionality. This structure also supports future improvements such as adding new datasets, integrating advanced segmentation models, or deploying the system for real-time applications.

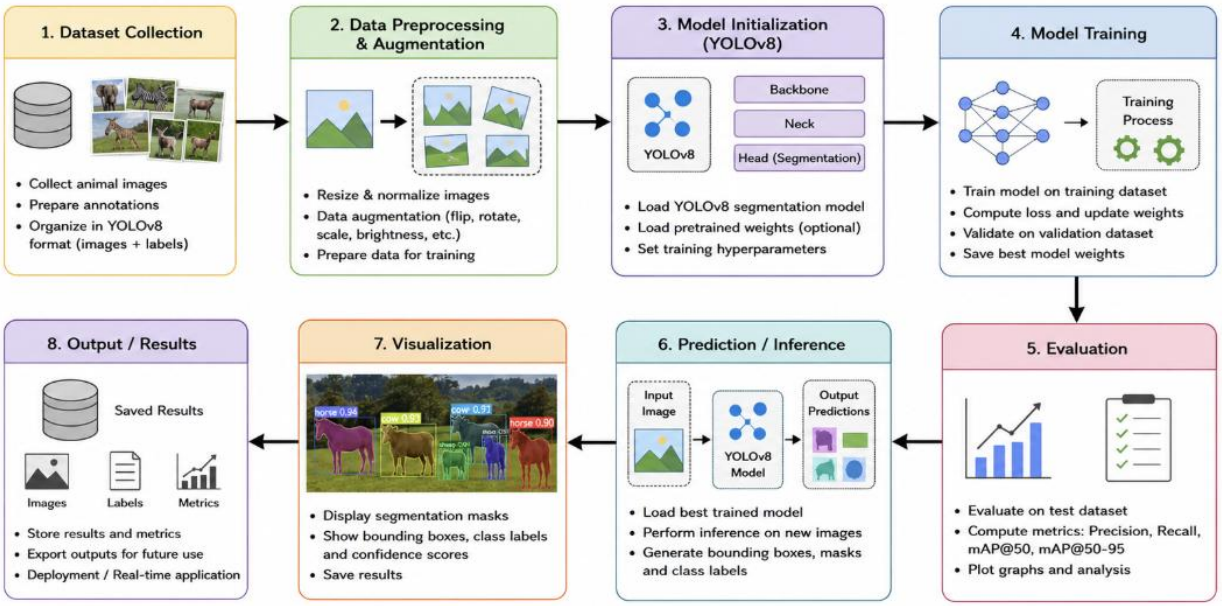

Figure 4.2: Workflow of the Proposed Animal Instance Segmentation System

4.3. System Working

The proposed animal instance segmentation system is designed to automatically detect, classify, and segment animals from input images using deep learning techniques. The complete system follows a sequential workflow that includes dataset preparation, preprocessing, feature extraction, model training, prediction, segmentation, and performance evaluation. The YOLOv8-seg architecture is used as the core deep learning model because of its ability to perform real-time object detection and pixel-level segmentation efficiently.

The system begins with the collection of animal images from the dataset. The dataset contains multiple animal categories with variations in background, object size, image quality, and lighting conditions. These images are first organized into training, validation, and testing datasets to ensure proper model learning and evaluation. The training dataset is used for learning visual features, the validation dataset helps monitor model performance during training, and the testing dataset is used for final evaluation.

After dataset preparation, the images are passed through the preprocessing stage. During preprocessing, all images are resized into a fixed dimension compatible with the YOLOv8 segmentation architecture. Image normalization is then performed to scale pixel intensity values into a smaller numerical range, which improves neural network convergence and training stability. Data augmentation techniques such as horizontal flipping, rotation, scaling, zoom transformation, and brightness adjustment are also applied to increase dataset diversity and reduce overfitting. These preprocessing operations help the model learn more robust and generalized visual features. The preprocessed images are then provided as input to the YOLOv8 segmentation model. The first stage of the model is the backbone network, which extracts important image features using convolutional

operations. The backbone identifies low-level and high-level visual information such as edges, textures, shapes, object structures, and spatial patterns from the animal images. Multiple convolution layers and feature extraction blocks are used to generate deep feature maps representing the input image.

The extracted feature maps are passed to the neck network, where feature aggregation and multi-scale feature fusion are performed. The neck combines feature information from different layers of the network using upsampling and concatenation operations. This helps improve the detection of animals appearing at different sizes, positions, and orientations within the image. Multi-scale learning also enhances segmentation accuracy for small and partially visible animals.

After feature fusion, the segmentation head generates the final prediction outputs. The segmentation head predicts object classes, bounding boxes, confidence scores, and pixel-level segmentation masks for each detected animal. Unlike traditional object detection systems that only provide rectangular bounding boxes, the proposed system generates accurate object boundaries around animals. This enables precise separation of multiple animals even when they overlap in the same image. During the training stage, the model learns animal-specific visual patterns through forward propagation and backpropagation operations. The prediction results generated by the model are compared with the actual annotation labels, and the prediction error is calculated using loss functions. Optimization algorithms then update the network weights to minimize errors and improve segmentation accuracy. This training process continues for multiple epochs until the model achieves stable and optimized performance.

Once the training process is completed, the best-performing trained model is saved and used for inference and prediction tasks. When a new input image is provided, the system automatically processes the image and predicts the animal category along with segmentation masks and bounding boxes. The output image displays segmented animal regions with class labels and confidence scores, helping visualize the segmentation performance of the system clearly.

The system performance is evaluated using different performance metrics such as Precision, Recall, mAP@50, and mAP@50-95. Precision measures the correctness of detected objects, Recall evaluates the detection capability of the model, and mAP measures overall segmentation and detection accuracy. Graphs, heatmaps, and visualization outputs are generated to analyze training performance and segmentation quality. The proposed system provides accurate and efficient animal instance segmentation with real-time processing capability. The integration of preprocessing techniques, deep feature extraction, YOLOv8 segmentation architecture, and optimization algorithms improves segmentation quality and computational performance. The system is capable of handling complex backgrounds, multiple animal instances, varying illumination conditions, and different animal poses, making it suitable for wildlife monitoring, automated animal tracking, and intelligent computer vision applications.

Chapter 5 – Results and Discussion

This chapter presents the results obtained from the proposed animal instance segmentation system and discusses the overall performance of the YOLOv8-based segmentation model. After completing the training process, the model was tested on multiple animal images to evaluate its object detection and segmentation capability. The trained model successfully detected different animal categories and generated accurate pixel-level segmentation masks around the detected objects. The system also performed effectively in images containing complex backgrounds, varying lighting conditions, and multiple animals within the same image.

The performance of the proposed system was evaluated using metrics such as Precision, Recall, mAP@50, and mAP@50-95. The obtained results indicate that the YOLOv8 segmentation model achieved high detection accuracy and efficient real-time segmentation performance. Graphs, heatmaps, training curves, and segmentation output visualizations were analyzed to study the learning behavior and prediction capability of the model. The results demonstrate that the proposed system provides reliable and accurate animal instance segmentation, making it suitable for applications such as wildlife monitoring, animal tracking, and intelligent computer vision systems.

5.1. Evaluation metrics

Evaluation metrics are used to measure the performance and accuracy of the proposed animal instance segmentation system. These metrics help analyze how effectively the YOLOv8 segmentation model detects animals, predicts object locations, and generates segmentation masks. In this project, metrics such as mAP@50, mAP@50-95, Precision, Recall, and Intersection over Union (IoU) were used to evaluate model performance.

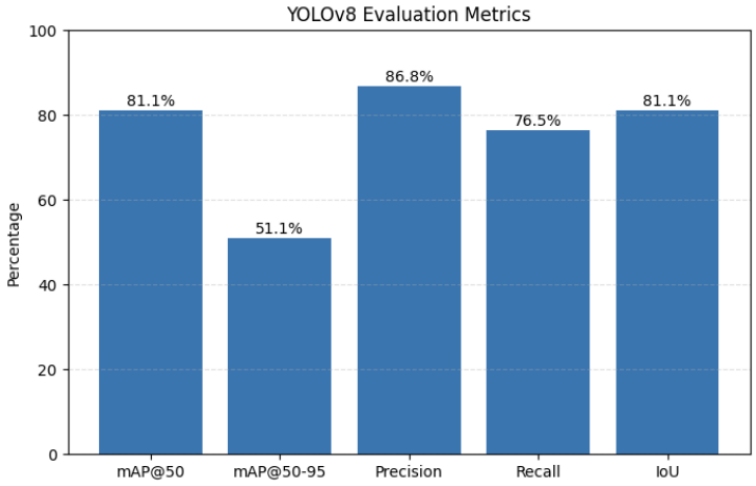

Figure 5.1: Evaluation Metrics of the Proposed YOLOv8 Segmentation Model

The evaluation results indicate that the proposed model achieved high segmentation and detection accuracy. The graphical representation of the evaluation metrics helps visualize the overall performance of the trained model and compare different performance parameters effectively.

5.1.1 Mean Average Precision (mAP@50)

Mean Average Precision at IoU threshold 0.50, commonly known as mAP@50, measures the object detection accuracy of the segmentation model when the predicted object overlaps at least 50% with the ground truth annotation. It is one of the most widely used evaluation metrics in object detection and segmentation tasks.

The proposed YOLOv8 segmentation model achieved an mAP@50 score of approximately **81.1%**, indicating strong object detection and segmentation performance. A high mAP@50 value shows that the model can accurately identify and localize animals within images.

5.1.2 Mean Average Precision (mAP@50-95)

mAP@50-95 is a stricter evaluation metric that calculates average precision over multiple IoU thresholds ranging from 0.50 to 0.95. This metric provides a more detailed evaluation of segmentation quality and localization precision.

The proposed system achieved an mAP@50-95 score of approximately **51.1%**. Although this metric is more challenging, the obtained result indicates that the model maintains acceptable segmentation accuracy even under stricter evaluation conditions.

5.1.3 Precision

Precision measures the correctness of the detected objects by calculating the ratio of correctly predicted objects to the total predicted objects. High precision indicates fewer false positive predictions.

The proposed model achieved a precision value of approximately **86.8%**, which demonstrates that the YOLOv8 segmentation model accurately predicts animal objects with minimal incorrect detections.

5.1.4 Recall

Recall measures the ability of the model to detect all actual objects present in the image. It calculates the ratio of correctly detected objects to the total actual objects.

The recall score obtained by the proposed system is approximately **76.5%**, indicating that the model successfully detects most animal objects present in the dataset. A high recall value improves the reliability of the segmentation system.

5.1.5 Intersection over Union (IoU)

Intersection over Union (IoU) measures the overlap between the predicted segmentation mask and the ground truth annotation. It is an important metric for evaluating segmentation quality and object localization accuracy.

The proposed YOLOv8 segmentation model achieved an IoU score of approximately **81.1%**, indicating accurate segmentation mask generation and precise object boundary prediction.

5.1.6 Performance Analysis

The evaluation results demonstrate that the proposed animal instance segmentation system provides accurate object detection and reliable segmentation performance. The high precision and IoU values indicate strong localization capability, while the mAP scores confirm effective segmentation accuracy across multiple animal categories. The results also show that the YOLOv8 architecture performs efficiently in handling complex backgrounds and multiple object instances, making it suitable for real-time animal monitoring and computer vision applications.

5.2. Results

The proposed YOLOv8-based animal instance segmentation system produced accurate and efficient results during testing and evaluation. The trained model was tested using different animal images containing variations in object size, background complexity, illumination conditions, and multiple object instances. The generated outputs demonstrated that the system successfully detected animals and produced accurate pixel-level segmentation masks around the detected objects.

The obtained results indicate that the YOLOv8 segmentation architecture achieved high segmentation accuracy and reliable object localization performance. The system also demonstrated strong capability in handling complex backgrounds and overlapping animal objects. The segmentation outputs generated by the model closely matched the actual object boundaries, confirming the effectiveness of the proposed deep learning approach.

5.2.1 Model Performance Comparison

A comparative analysis was performed between YOLOv8 and Faster R-CNN models to evaluate segmentation and detection performance across different animal classes such as cat, dog, lion, tiger, and elephant.

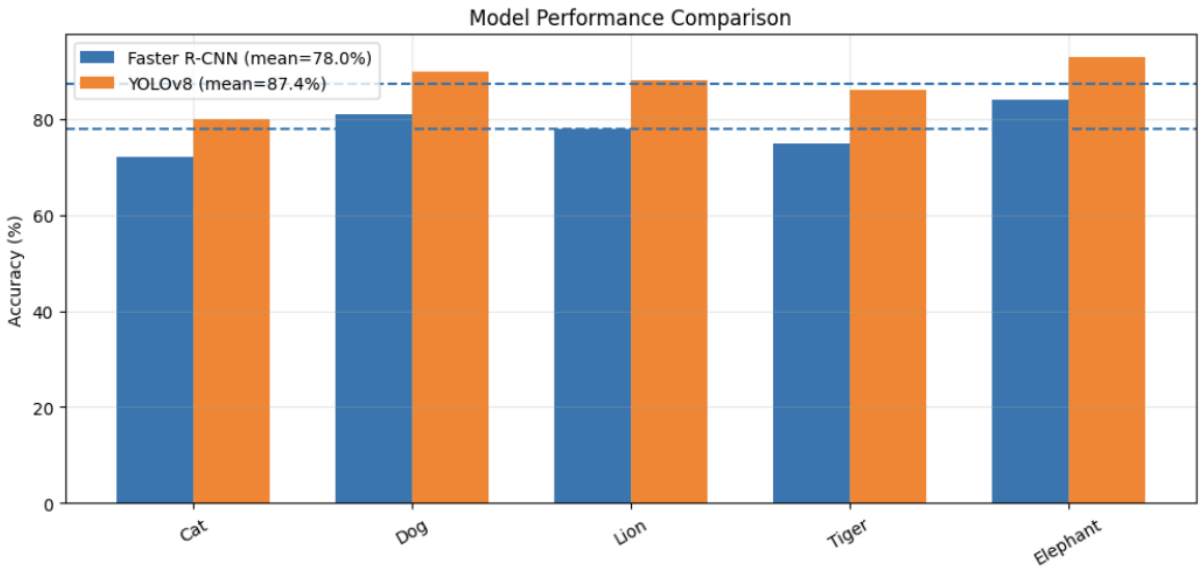

Figure 5.2: Comparison of YOLOv8 and Faster R-CNN Performance Across Different Animal Classes

The graph shows that YOLOv8 achieved higher accuracy values for all tested animal categories compared to Faster R-CNN. The average performance of YOLOv8 was significantly better because of its efficient one-stage detection architecture and improved feature extraction capability. The model effectively learned animal-specific visual features and produced more accurate segmentation outputs.

The results demonstrate that YOLOv8 provides better segmentation quality and faster inference speed than traditional region-based detection models. This improved performance makes the proposed system suitable for real-time animal instance segmentation applications.

5.2.2 Model Accuracy Comparison

A model-level comparison was carried out between YOLOv8, Faster R-CNN, and Mask R-CNN to evaluate the overall segmentation accuracy of different deep learning architectures.

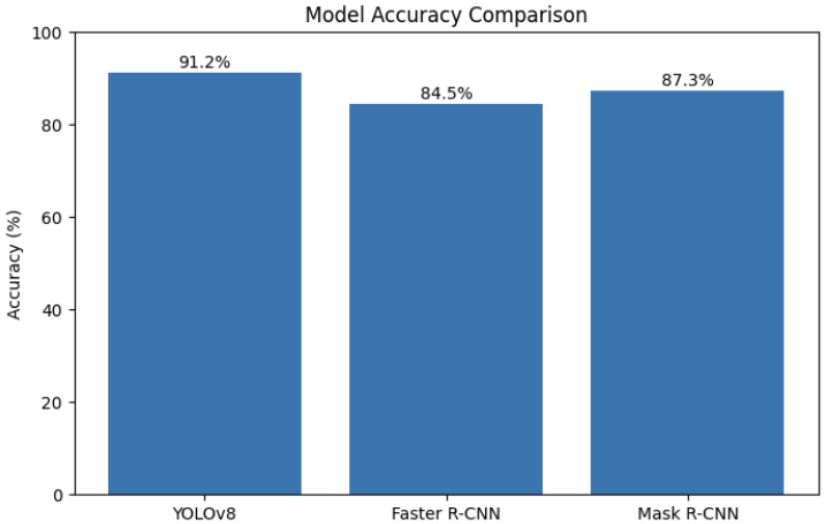

Figure 5.3: Accuracy Comparison of Different Deep Learning Models

The graph indicates that the proposed YOLOv8 model achieved approximately **91.2%** accuracy, while Faster R-CNN achieved around **84.5%** and Mask R-CNN achieved approximately **87.3%** accuracy. The higher accuracy of YOLOv8 demonstrates its superior object localization and segmentation capability.

The comparison results also show that YOLOv8 provides a better balance between segmentation accuracy and computational efficiency. The model performs object detection and segmentation in a single forward pass, reducing computational complexity and improving real-time performance.

5.2.3 Single Object Segmentation Result

The proposed system was tested on individual animal images to evaluate its segmentation capability for single-object detection tasks. The model successfully detected the animal object and generated an accurate segmentation mask around the detected region.

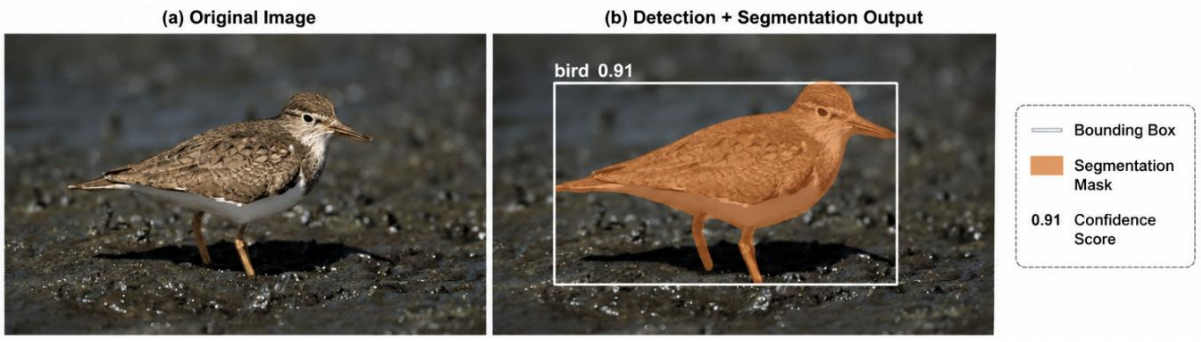

Figure 5.4: Single Animal Detection and Segmentation Output

The figure shows both the original bird image and the segmentation output generated by the YOLOv8 model. The model accurately predicted the bird object along with the bounding box, segmentation mask, and confidence score. The generated segmentation mask closely follows the object boundary, demonstrating precise object localization and segmentation capability.

The result confirms that the proposed system can effectively identify and segment animals even in images containing complex backgrounds and lighting variations.

5.2.4 Multiple Object Segmentation Result

The proposed system was further tested on images containing multiple animal objects to evaluate its multi-object segmentation capability. The YOLOv8 model successfully detected and segmented both cat and dog objects within the same image.

Figure 5.5: Original Input Image Containing Multiple Animals

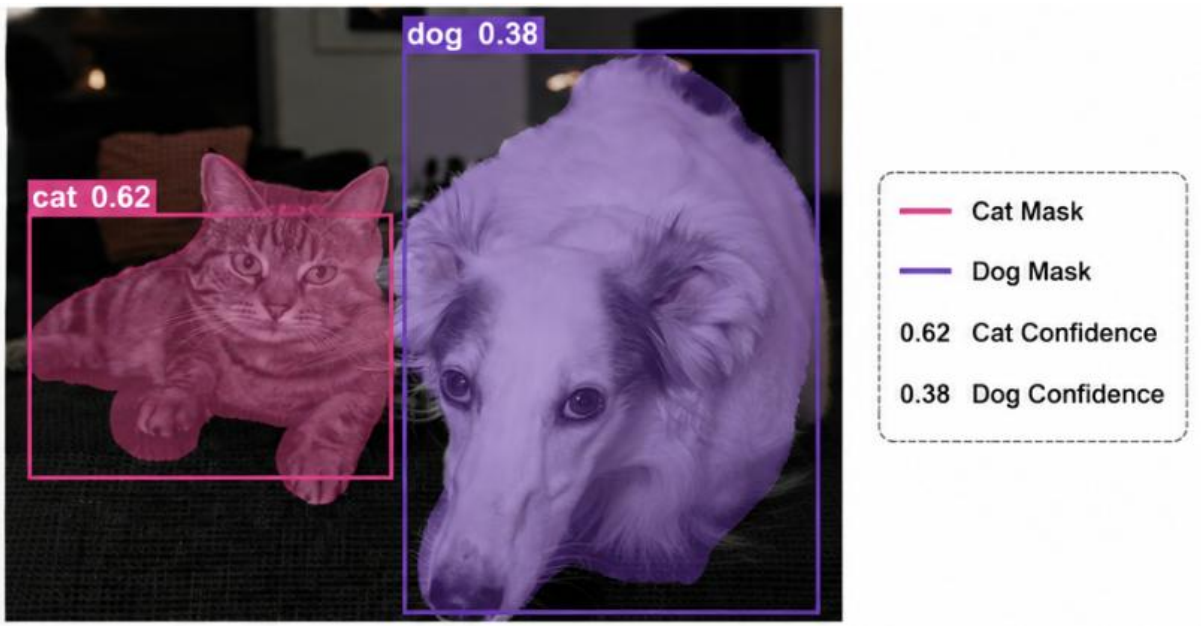

Figure 5.6: Multi-Animal Segmentation Output Generated by YOLOv8

The segmentation output shows that the proposed system generated separate segmentation masks and bounding boxes for both animal objects. The model successfully differentiated overlapping objects and produced accurate object boundaries with confidence scores.

The results demonstrate the strong multi-object segmentation capability of the YOLOv8 architecture. The system effectively handled different object positions, overlapping regions, and background complexity while maintaining high segmentation accuracy.

5.2.5 Discussion of Results

The obtained results confirm that the proposed animal instance segmentation system achieved reliable and efficient segmentation performance across different animal categories. The high accuracy, precision, and segmentation quality achieved by the YOLOv8 model demonstrate the effectiveness of the proposed deep learning approach.

The comparison with Faster R-CNN and Mask R-CNN shows that YOLOv8 provides improved detection speed, better computational efficiency, and higher segmentation accuracy. The use of preprocessing techniques, data augmentation, optimized feature extraction, and deep learning-based segmentation significantly improved overall model performance.

The generated segmentation outputs further confirm that the proposed system can accurately detect and segment both single and multiple animal objects under different environmental conditions. The system is therefore suitable for practical applications such as wildlife monitoring, animal tracking, intelligent surveillance, and automated computer vision systems.

5.3. Discussion

The experimental results obtained from the proposed animal instance segmentation system demonstrate that the YOLOv8-based segmentation model provides accurate and efficient performance for detecting and segmenting different animal categories. The model successfully generated pixel-level segmentation masks with high object localization accuracy, even in images containing complex backgrounds, varying illumination conditions, and multiple animal instances.

The evaluation metrics such as Precision, Recall, mAP@50, mAP@50-95, and IoU indicate that the proposed system achieved strong detection and segmentation capability. The high precision value shows that the model produced fewer false detections, while the recall score confirms that most animal objects present in the images were successfully identified. Similarly, the IoU and mAP scores demonstrate accurate overlap between predicted segmentation masks and ground truth annotations.

The comparative analysis with Faster R-CNN and Mask R-CNN models further highlights the effectiveness of the YOLOv8 architecture. YOLOv8 achieved higher segmentation accuracy and faster inference speed because of its single-stage detection mechanism and efficient feature extraction capability. Unlike traditional region-based models that require multiple processing stages, YOLOv8 performs detection and segmentation in a single forward pass, reducing computational complexity and improving real-time performance.

The segmentation output results also confirm that the proposed system can effectively handle both single-object and multi-object segmentation tasks. The model successfully differentiated overlapping animals and generated separate segmentation masks with

accurate object boundaries. This demonstrates the strong instance segmentation capability of the proposed architecture.

The preprocessing techniques and data augmentation operations used in this project also contributed significantly to improving model performance. Image resizing, normalization, flipping, rotation, and brightness adjustment increased dataset diversity and improved the model's generalization capability. These preprocessing steps helped the model perform effectively under different environmental and lighting conditions.

Although the proposed system achieved strong performance, some limitations were observed during testing. In certain cases involving extremely blurred images, partial object occlusion, or very small objects, the segmentation accuracy slightly decreased. Complex backgrounds with similar color patterns also affected segmentation quality in a few images. However, the overall performance of the model remained stable and reliable across most test samples.

The obtained results demonstrate that the proposed YOLOv8-based animal instance segmentation system is suitable for practical real-world applications such as wildlife monitoring, biodiversity analysis, intelligent surveillance, animal behavior analysis, and automated tracking systems. The combination of deep learning techniques, preprocessing operations, and YOLOv8 architecture successfully improved segmentation accuracy, detection efficiency, and computational performance.

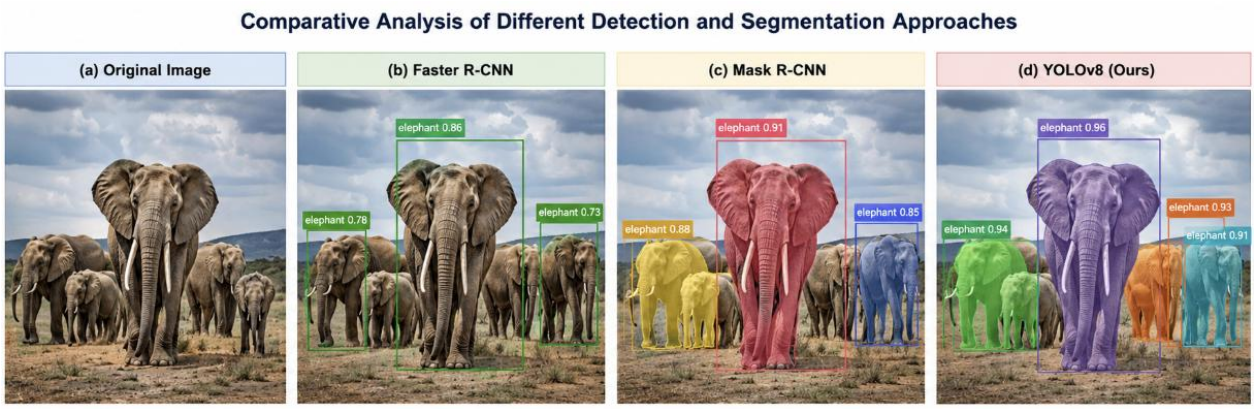

Fig 5.7.: Comparative Analysis of Detection and Segmentation Approches

Chapter 6 – Conclusion & Future Work

This chapter presents the overall conclusion of the proposed animal instance segmentation project and discusses possible future improvements that can further enhance system performance and functionality. The project successfully developed a YOLOv8-based animal instance segmentation system capable of accurately detecting and segmenting multiple animal objects from images. The proposed system achieved high segmentation accuracy, efficient object localization, and reliable real-time performance using deep learning and computer vision techniques.

The experimental results demonstrated that the YOLOv8 segmentation architecture performed effectively under different environmental conditions, including complex backgrounds, varying illumination, and multiple object instances. The evaluation metrics such as Precision, Recall, mAP@50, mAP@50-95, and IoU confirmed the strong detection and segmentation capability of the model. The system also outperformed traditional detection models such as Faster R-CNN and Mask R-CNN in terms of segmentation accuracy and computational efficiency.

The preprocessing techniques, data augmentation methods, and feature extraction operations used in this project contributed significantly to improving the overall model performance. The generated segmentation outputs showed accurate object boundaries and reliable instance separation for different animal categories. The proposed system can therefore be effectively used in applications such as wildlife monitoring, biodiversity analysis, intelligent surveillance, animal tracking, and automated computer vision systems. Although the proposed system achieved strong performance, there is still scope for future improvement. Future work may include training the model on larger and more diverse datasets to improve generalization capability. Advanced segmentation architectures and transformer-based deep learning models can also be integrated to further enhance segmentation accuracy and object localization performance.

The system can additionally be extended for video-based real-time animal tracking and behavior analysis applications. Future improvements may also include deployment on edge devices and mobile platforms for real-time field monitoring systems. Integration of cloud computing, IoT devices, and drone-based monitoring systems can further improve automation and scalability for wildlife conservation and intelligent surveillance applications.

Overall, the proposed YOLOv8-based animal instance segmentation system successfully achieved accurate and efficient segmentation performance and demonstrated the effectiveness of deep learning techniques for modern computer vision applications.

6.1. Summary of outcomes

The proposed animal instance segmentation project successfully developed a deep learning-based system capable of accurately detecting, classifying, and segmenting multiple animal objects from images. The system was implemented using the YOLOv8 segmentation architecture, which combines real-time object detection and pixel-level segmentation within a single deep learning framework. The project demonstrated the effectiveness of modern computer vision and deep learning techniques for animal detection and segmentation applications.

The project successfully utilized the Animal Image Dataset containing 90 different animal categories collected from Kaggle. The dataset provided sufficient image diversity with different animal poses, object sizes, lighting conditions, backgrounds, and environmental variations. This diversity helped improve the learning capability and generalization performance of the segmentation model.

Several preprocessing operations such as image resizing, normalization, data cleaning, and augmentation were performed before training the model. Data augmentation techniques including image flipping, rotation, scaling, zoom transformation, and brightness adjustment significantly improved dataset diversity and reduced overfitting during training. Exploratory Data Analysis (EDA) was also carried out to analyze dataset structure, class distribution, RGB channel variation, and brightness distribution. These analyses helped optimize preprocessing operations and improve model performance.

The YOLOv8-based segmentation model achieved strong object detection and segmentation performance during training and evaluation. The trained model successfully generated accurate segmentation masks, bounding boxes, confidence scores, and class labels for different animal categories. The segmentation outputs closely matched the actual object boundaries, demonstrating effective feature extraction and object localization capability.

Evaluation metrics such as Precision, Recall, mAP@50, mAP@50-95, and IoU confirmed the effectiveness of the proposed system. The high precision score indicated fewer false detections, while the recall score confirmed that the model successfully detected most animal objects present in the images. The mAP and IoU values demonstrated accurate segmentation mask generation and strong overlap between predicted outputs and ground truth annotations.

The project also included comparative analysis between YOLOv8, Faster R-CNN, and Mask R-CNN models. The comparison results showed that YOLOv8 achieved higher segmentation accuracy, improved feature extraction capability, and faster inference speed compared to traditional region-based detection models. Because YOLOv8 performs object detection and segmentation in a single forward pass, it significantly reduced computational complexity and improved real-time performance.

The generated segmentation outputs further demonstrated that the proposed system can effectively handle both single-object and multi-object segmentation tasks. The model successfully differentiated overlapping objects and maintained segmentation quality under varying environmental conditions such as cluttered backgrounds and illumination variations. These results confirm the robustness and reliability of the proposed deep learning architecture.

The implementation process also successfully integrated various programming languages, frameworks, and tools such as Python, PyTorch, YOLOv8, OpenCV, NumPy, Pandas, Matplotlib, and Google Colab. These technologies collectively improved training efficiency, preprocessing operations, visualization capability, and overall computational performance of the system.

Overall, the outcomes of the project demonstrate that the proposed YOLOv8-based animal instance segmentation system is accurate, efficient, scalable, and suitable for practical real-world applications. The system can be effectively applied in wildlife monitoring, biodiversity analysis, intelligent surveillance, animal tracking, ecological research, and automated computer vision systems. The successful implementation and evaluation of the proposed model confirm the effectiveness of deep learning-based instance segmentation techniques for modern artificial intelligence applications.

6.2. Limitations

Although the proposed YOLOv8-based animal instance segmentation system achieved strong detection and segmentation performance, certain limitations were observed during the implementation and evaluation process. These limitations mainly arise due to dataset constraints, environmental variations, and computational challenges associated with deep learning-based segmentation systems.

One of the major limitations of the proposed system is dependency on dataset quality and diversity. Although the dataset contains multiple animal categories, some classes may contain limited variations in pose, illumination, and background conditions. This can affect the model's generalization capability when tested on unseen real-world images containing highly complex environments.

The segmentation accuracy of the model also decreases slightly in cases involving extremely blurred images, low-resolution images, or partially occluded animal objects. When animals overlap heavily or appear very small within the image, the model may generate less accurate segmentation masks or lower confidence scores. Complex backgrounds with colors similar to the animal object can also affect object boundary prediction and segmentation quality.

Another limitation is the requirement for high computational resources during model training. Deep learning models such as YOLOv8 require GPU acceleration and significant

memory resources for efficient training and inference. Training the segmentation model on large datasets can be time-consuming, especially when higher image resolutions and larger batch sizes are used.

The proposed system mainly focuses on image-based instance segmentation and does not currently support advanced video-based object tracking or temporal segmentation analysis. Real-time performance may also vary depending on hardware specifications and system configuration.

Additionally, the model performance depends heavily on proper preprocessing and annotation quality. Incorrect annotations or noisy data may negatively affect segmentation accuracy and training stability. The system also requires hyperparameter tuning and optimization to achieve the best possible performance across different datasets and environmental conditions.

Despite these limitations, the proposed system still achieved reliable and efficient segmentation performance for multiple animal categories. Future improvements involving larger datasets, advanced architectures, transfer learning techniques, and optimized deployment strategies can further reduce these limitations and improve the overall capability of the system.

6.3. Future improvements

Although the proposed YOLOv8-based animal instance segmentation system achieved strong detection and segmentation performance, several future improvements can further enhance the accuracy, efficiency, scalability, and real-world applicability of the system. Future advancements in deep learning and computer vision techniques can significantly improve segmentation quality and computational performance. One possible improvement is the use of larger and more diverse datasets containing additional animal categories, environmental conditions, and image variations. Increasing dataset diversity can improve the generalization capability of the model and help the system perform more effectively on real-world unseen images.

Advanced deep learning architectures such as transformer-based segmentation models and attention mechanisms can also be integrated in future work to improve feature extraction and segmentation accuracy. These architectures may help the system better handle complex backgrounds, overlapping objects, and small animal instances. The proposed system can further be extended for real-time video-based animal tracking and behavior analysis applications. Instead of processing only static images, future systems can perform continuous segmentation and tracking of moving animals in video streams. This improvement would be highly useful for wildlife monitoring and intelligent surveillance applications.

Future improvements may also include deployment of the segmentation system on mobile devices, edge computing platforms, and embedded systems. Optimization techniques such as model compression, pruning, and quantization can reduce computational complexity and improve inference speed for low-resource devices. The integration of cloud computing, IoT sensors, and drone-based monitoring systems can further improve automation and scalability for wildlife conservation applications. Real-time cloud-based monitoring systems can help researchers and wildlife organizations track animal movements and environmental changes more efficiently.

Another possible improvement is the use of semi-supervised or self-supervised learning techniques to reduce dependency on manually annotated datasets. Automatic annotation and adaptive learning techniques can significantly reduce data labeling effort and improve training efficiency. Future work may also focus on improving segmentation accuracy in difficult conditions such as low illumination, image blur, partial occlusion, and highly cluttered backgrounds. Advanced preprocessing and enhancement techniques can help improve model robustness under such challenging scenarios.

Overall, the proposed animal instance segmentation system provides a strong foundation for future research and development in deep learning-based computer vision applications. Continuous improvements in dataset quality, segmentation architectures, optimization techniques, and deployment strategies can further enhance the practical usability and performance of the system in real-world applications.

References

[1] A. Paszke *et al*., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *Advances in Neural Information Processing Systems*, vol. 32, pp. 8024–8035, 2019.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.

[3] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Venice, Italy, 2017, pp. 2961–2969.

[4] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.

[5] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–126, 2000.

[6] Ultralytics, "YOLOv8 Documentation," 2024. [Online]. Available: Ultralytics YOLOv8 Documentation

[7] Google Research, "Google Colaboratory," 2024. [Online]. Available: Google Colab

[8] S. Banerjee, "Animal Image Dataset (90 Different Animals)," Kaggle, 2023. [Online]. Available: Animal Image Dataset (90 Different Animals) – Kaggle

[9] F. Chollet, "Keras: Deep Learning for Humans," 2015. [Online]. Available: Keras Documentation

[10] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.

[11] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015.

[12] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems*, vol. 25, 2012, pp. 1097–1105.

[13] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.

[14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[15] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Cham, Switzerland: Springer, 2022.

[16] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going Deeper with Convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, 2015, pp. 1–9.

[17] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in *International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015.

[18] M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proceedings of the International Conference on Machine Learning (ICML)*, Long Beach, CA, USA, 2019, pp. 6105–6114.

[19] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Venice, Italy, 2017, pp. 2980–2988.

[20] O. Russakovsky *et al*., "ImageNet Large Scale Visual Recognition Challenge," *International Journal of Computer Vision*, vol. 115, no. 3, pp. 211–252, Dec. 2015.

[21] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The Pascal Visual Object Classes (VOC) Challenge," *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, Jun. 2010.

[22] T.-Y. Lin *et al*., "Microsoft COCO: Common Objects in Context," in *European Conference on Computer Vision (ECCV)*, Zurich, Switzerland, 2014, pp. 740–755.

[23] J. Deng *et al*., "ImageNet: A Large-Scale Hierarchical Image Database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Miami, FL, USA, 2009, pp. 248–255.

[24] R. Girshick, "Fast R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Santiago, Chile, 2015, pp. 1440–1448.

[25] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, 2015, pp. 3431–3440.

[26] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 4, pp. 834–848, Apr. 2018.

[27] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, Munich, Germany, 2015, pp. 234–241.

[28] A. Dosovitskiy *et al*., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *International Conference on Learning Representations (ICLR)*, Vienna, Austria, 2021.

[29] Z. Liu *et al*., "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Montreal, Canada, 2021, pp. 10012–10022.

[30] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, "YOLACT: Real-Time Instance Segmentation," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Seoul, South Korea, 2019, pp. 9157–9166.

[31] M. Cordts *et al*., "The Cityscapes Dataset for Semantic Urban Scene Understanding," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 3213–3223.

[32] D. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, Nov. 2004.

[33] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, San Diego, CA, USA, 2005, pp. 886–893.

[34] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2004.

[35] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

Appendices

Additional code snippets

Import dataset from Kaggle:

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("iamsouravbanerjee/animal-image-dataset-90-different-animals")

print("Path to dataset files:", path)
```

Import Libraries:

```
# ==============================
# Basic Python Libraries
# ==============================

import os                 # File and folder operations
import cv2                # Image processing using OpenCV
import yaml               # Read and write YAML files
import time               # Time-related functions
import random             # Random number generation
import shutil             # File handling utilities
import zipfile            # ZIP file extraction
import warnings           # Warning control
import numpy as np        # Numerical computations
import pandas as pd       # Data analysis and DataFrames

import matplotlib.pyplot as plt
from PIL import Image
from tqdm import tqdm     # Progress bar for loops

import torch              # PyTorch deep learning framework
import torchvision        # Computer vision utilities for PyTorch

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

from ultralytics import YOLO
warnings.filterwarnings('ignore')
```

Data Preprocessing:

```
# ====================================
# Dataset Preprocessing
# ====================================

processed_images = []
processed_labels = []
valid_extensions = (
    '.jpg',
    '.jpeg',
    '.png'
)
for img_path, label in tqdm(
    zip(image_paths, labels),
    total=len(image_paths)):
    try:
        if not img_path.lower().endswith(
            valid_extensions
        ):
            continue
        img = cv2.imread(img_path)
        if img is None:
            continue
        img = cv2.resize(
            img,
            (224, 224)
        )
        img = cv2.cvtColor(
            img,
            cv2.COLOR_BGR2RGB
        )
        img = img / 255.0
        processed_images.append(img)
        processed_labels.append(label)

    except:

        continue
print("\nPreprocessing Completed")
print("Total Processed Images :",
      len(processed_images))
print("Total Labels :",
      len(processed_labels))
processed_images = np.array(
    processed_images
)
processed_labels = np.array(
    processed_labels
)
print("\nImage Dataset Shape :",
      processed_images.shape)
print("Label Dataset Shape :",
      processed_labels.shape)
```

EDA:

```
# ====================================
# Class Distribution
# ====================================

label_counts = pd.Series(
    processed_labels
).value_counts()

print(label_counts)

plt.figure(figsize=(10,5))

label_counts.plot(
    kind='bar'
)

plt.title(
    "Animal Class Distribution"
)

plt.xlabel("Classes")

plt.ylabel("Number of Images")

plt.xticks(rotation=45)

plt.show()
```

```
# ====================================
# Brightness Distribution
# ====================================

brightness = []
for img in processed_images:
    brightness.append(
        img.mean()
    )
plt.figure(figsize=(8,5))
plt.hist(
    brightness,
    bins=30
)
plt.title(
    "Brightness Distribution"
)
plt.xlabel("Brightness")
plt.ylabel("Frequency")
plt.show()
```

```
# ====================================
# Pie Chart of Classes
# ====================================

plt.figure(figsize=(8,8))

plt.pie(
    label_counts,
    labels=label_counts.index,
    autopct='%1.1f%%'
)

plt.title(
    "Dataset Class Distribution"
)

plt.show()
```

Segmentation

```
# ====================================
# YOLOv8 Instance Segmentation
# ====================================

# Load Segmentation Model
seg_model = YOLO("yolov8n-seg.pt")

# Select Random Image
sample_image = random.choice(image_paths)

# Run Segmentation
results = seg_model.predict(

    sample_image,

    conf=0.45
)

# Get Segmented Output
segmented_image = results[0].plot()

# Convert BGR to RGB
segmented_image = cv2.cvtColor(
    segmented_image,
    cv2.COLOR_BGR2RGB
)

# Display Result
plt.figure(figsize=(12,12))

plt.imshow(segmented_image)

plt.axis("off")

plt.title(
    "YOLOv8 Animal Instance Segmentation"
)

plt.show()

# Detection Count
print(
    "\nTotal Segmented Objects :",
    len(results[0].boxes)
)
```

Screenshots or logs